LabGuard: An Environmental Drift and Storage Monitoring System

Project Proposal for UCS503P
Submitted to: Paramveer Sidhu
Thapar Institute of Engineering and Technology

Aditi Sinha, CSED* Sajneet Kaur Grewal, CSED†

August 16, 2026

1 Higher-order goal

. . . secondary framing

This project aims to improve research reproducibility, lab safety, and asset integrity in university facilities by preventing sample and reagent degradation. While environmental monitoring is applicable across industrial facilities, the immediate engineering objective is to translate *continuous compliance and drift alerting* into a measurable, lightweight, and deployable software pipeline.

2 Time-to-value

. . . primary heuristic

- We will deliver meaningful increments quickly by prototyping the core ingestion and violation-checking workflow within a 2-week iteration window.
- We will prioritize fast feedback over complex infrastructure by using weekly iteration cycles and automated CI testing to catch defects early.
- Success is defined by what can be ingested, validated, and visually flagged in the first iteration, and incrementally expanded thereafter.

3 Problem Statement

In campus research laboratories and department storerooms, temperature-sensitive chemical reagents, biological cultures, and ongoing experiments often suffer irreversible damage due to undetected environmental drift (e.g., refrigeration failure, door ajar, HVAC anomalies). Common pain points include:

- **Manual and sporadic logging:** measurements are often recorded on physical paper logs once a day, missing rapid transient failures.
- **Delayed incident visibility:** lab technicians discover spoilage or condition violations only after experiments fail or samples degrade.

*Roll No: 1024030xxx, Email: asinha_be24thapar.edu
†Roll No: 1024030xxx, Email: sgrewal_be24thapar.edu

- **Absence of centralized thresholds:** different storage units require distinct tolerance bounds, leading to operator error.
- **Lack of audit trails:** no queryable history exists to prove whether an experiment remained within compliance specifications.

Why this matters now (contextual relevance): In academic research laboratories with high sample turnover and shared equipment, automated condition monitoring prevents costly material loss, saves experiment runtime, and reduces hazardous chemical degradation.

4 Proposed Solution *... Software Proposition*

4.1 Overview

Build a **web-based** "Lab Environmental Monitoring" platform with the following components:

- **Telemetry ingestion API:** accepts periodic sensor payloads (sensor ID, temperature, humidity, timestamp).
- **Deterministic rule evaluator:** automatically checks incoming readings against unit-specific minimum/maximum threshold boundaries.
- **Live monitor dashboard:** displays active storage unit health status with clear visual badges (Safe vs. Warning/Alert).
- **Historical log viewer:** visualizes recent reading trends to spot gradual thermal or humidity drift.
- **Incident logging:** automatically records timestamped threshold breaches for compliance auditing.

4.2 Core workflow

1. Edge sensor (or test simulation script) sends an HTTP payload to the ingestion endpoint.
2. Backend validates payload structure and compares values against configured threshold envelopes.
3. Reading is persisted in the database alongside its evaluated status ("SAFE" or "ALERT").
4. Dashboard updates to reflect live status, rendering immediate visual warnings if a threshold is breached.

4.3 Operational constraints for an educational, campus setting

- Keep the system lightweight and responsive on standard desktop/mobile browsers.
- Avoid reliance on complex machine learning or distributed stream engines; focus on deterministic correctness, automated testing, and clear workflow transitions.
- Design for straightforward deployment using standard relational storage and basic cloud/local hosting.

5 Solution Approach *... Engineering focus*

This is an engineering course project, so we focus on software correctness, data integrity, and pipeline automation rather than experimental algorithms.

5.1 Threshold evaluation *... non-research*

Threshold checking will be deterministic and rule-based:

- Standardize payload structures with strict data type validation (numeric bounds, non-null values).
- Evaluate readings using standard boundary comparison logic based on material safety tolerance tables.
- Classify readings into distinct states (Normal, Warning, Critical) without black-box models.

5.2 Web architecture *... web-based solution*

A standard 3-tier web architecture:

- **Frontend:** responsive web dashboard to view live storage unit statuses, history charts, and alerts.
- **Backend API:** REST endpoints for telemetry ingestion, threshold configuration, and log retrieval.
- **Database:** relational database (SQLite/PostgreSQL) storing sensor metadata, threshold definitions, telemetry logs, and incident records.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically run unit tests verifying threshold arithmetic, input validation, and boundary conditions on every push.
- Ensure payload schema regressions are caught before merging to the main branch.
- Automate packaging and deployment to a staging environment.

6 Evaluation Criterion *... measurable and attributable*

We define success using measurable metrics tied to the core workflow.

6.1 Primary evaluation metric

Alert Trigger Latency (ATL): time elapsed between the receipt of an out-of-bounds telemetry payload and the generation of an alert state in the system.

- Target: median ATL ≤ 1 second during pilot testing.
- Attribution: measured from payload receipt timestamp to alert record commit timestamp in the database.

6.2 Secondary evaluation metrics

- **Boundary evaluation accuracy:** 100% deterministic accuracy on test suites covering edge cases ($T = T_{max}$, $T > T_{max}$, and invalid input types).
- **Input rejection rate:** 100% clean rejection (HTTP 400) of malformed payloads without server crashes.
- **UI status freshness:** live dashboard status reflection within 2 seconds of new telemetry ingestion.
- **Pipeline reliability:** $\geq$ 99% test suite pass rate in CI prior to release.

6.3 Pilot validation plan *... high-level*

- Run automated telemetry simulation scripts replicating normal operating conditions and simulated refrigeration failure spikes.
- Validate with 2–3 lab student scenarios (setting thresholds for cold storage vs. ambient chemical cabinets) and evaluate clarity of visual alerts.

7 Scalability *... with foresight*

The proposition is designed to scale systematically in both theoretical and practical dimensions.

1. **Scalable (theoretically):** The system supports scaling to multiple campus departments and units by:
   - decoupling telemetry ingestion endpoints from dashboard presentation layers,
   - applying database indexing on timestamp and sensor ID fields for fast time-series queries,
   - utilizing stateless API design for concurrent request handling.
2. **Operationally deployable (practically):** The system runs reliably on standard infrastructure:
   - relational database backing,
   - containerized packaging for consistent execution across environments,
   - GitHub Actions CI pipeline for automated testing and continuous integration.

8 Engine availability heuristic

A mature set of software components is available to implement this without research bottlenecks:

- Web frameworks for REST APIs (e.g., FastAPI, Express.js).
- Standard relational database (e.g., PostgreSQL, SQLite).
- Frontend rendering and visualization libraries (e.g., React, Chart.js).
- Automated testing frameworks (e.g., Pytest, Jest).
- CI runner (e.g., GitHub Actions) for automated test execution.

These mature tools enable the team to focus on input reliability, deterministic testing, and clear UX rather than low-level infrastructure development.

9 Project scope and deliverables *. . . iteration-friendly*

9.1 Initial deliverable *. . . first iteration*

- Ingestion endpoint (`POST /api/telemetry`) with basic schema validation
- Deterministic threshold comparison logic for incoming values
- Single-table relational schema storing readings and alert flags
- Basic frontend status view displaying latest sensor reading and a Safe/Alert indicator
- CI pipeline: automated linting and unit tests verifying threshold arithmetic

9.2 Subsequent deliverables

- Interactive historical trend chart for temperature and humidity
- Configuration UI to define custom Min/Max thresholds per sensor unit
- Automated simulation script generating periodic test traffic and anomaly spikes
- Outbound notification integration (email/webhook) for critical alerts
- CSV export for historical compliance audit records

10 Risks and mitigations

- **Malformed sensor data:** implement strict schema validation and type checking at the API boundary to reject bad inputs safely.
- **False positive alarms from transient sensor noise:** enforce configurable tolerance windows (e.g., requiring 2 consecutive out-of-bound readings) in subsequent iterations.
- **Evaluation measurement ambiguity:** instrument explicit timestamps on ingestion and persistence to clearly measure alert trigger latency.
- **Scope creep:** maintain a strict separation between core telemetry workflows and optional notification integrations.

11 Summary

This project proposes a lightweight, deployable web platform to monitor environmental parameters and prevent reagent degradation in campus laboratories. It meets the course criteria by emphasizing rapid delivery through a functional 2-week MVP, using CI automation to ensure threshold evaluation correctness, and tracking measurable performance metrics (alert trigger latency and boundary evaluation accuracy). It maintains a strict engineering focus on workflow robustness and reliability rather than unproven experimental algorithms.